

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de TADs**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 04/05/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de **Entre Ríos**

Problema N° 3 Algoritmos de Ordenamiento en Python

2. INTRODUCCIÓN

En este trabajo implementamos y comparamos tres algoritmos de ordenamiento en Python: **Burbuja**, **Quicksort** y **Radix Sort**. Para cada uno explicamos cómo funciona, mostramos el código con sus partes clave y justificamos la complejidad temporal en los distintos escenarios posibles. La idea es entender no solo que funcionan, sino por qué cada uno tiene sentido en ciertos contextos. **Burbuja** nos sirve como punto de referencia de un algoritmo simple pero costoso. **Quicksort** es el que usaríamos en la práctica para uso general. Y **Radix Sort** nos muestra que hay formas de ordenar que directamente evitan las comparaciones, aprovechando la estructura numérica de los datos.

3. DESARROLLO

A. Análisis y planteo

El problema nos pide evaluar distintos enfoques de ordenamiento para entender cuál conviene según la naturaleza de los datos. Los tres algoritmos que elegimos cubren espectros muy distintos:

1. **Burbuja:** Algoritmo de comparación simple. Lo incluimos como referencia: es fácil de entender, pero tiene complejidad $O(n^2)$ y no escala bien.
2. **Quicksort:** División y conquista con pivote aleatorio. Es la opción de uso general con $O(n \log n)$ promedio. El pivote aleatorio es clave para evitar el peor caso en listas ya ordenadas.
3. **Radix Sort:** Algoritmo no comparativo que ordena dígito a dígito. Asumimos que los elementos son enteros no negativos. Es $O(n \cdot k)$ determinista, ideal para grandes volúmenes numéricos con rango acotado.



Tabla de Pre y Postcondiciones:

Algoritmo	Precondición	Postcondición
Burbuja	Lista con elementos comparables entre sí.	Lista ordenada de forma ascendente in-place.
Quicksort	Lista inicializada y definición de índices de partición.	Segmentos de lista ordenados recursivamente mediante pivoteo.
Radix Sort	Lista de números enteros no negativos.	Lista ordenada basada en la distribución por dígitos (LSD).

B. Implementación / Resolución

Burbuja

El algoritmo compara elementos adyacentes e intercambia sus posiciones si están desordenados. Hace varias pasadas sobre la lista hasta que queda ordenada: en cada pasada, el valor más grande "burbujea" hacia el final. Implementamos una optimización que corta el proceso si en una pasada no hubo ningún intercambio, lo que significa que la lista ya está ordenada.

Método ordenar_lista() ($O(n^2)$): Usa un bucle externo (**while**) para controlar las pasadas y uno interno (**for**) para las comparaciones e intercambios. El flag **esta_ordenada** permite cortar antes si la lista ya no necesita más pasadas.

```
class Burbuja:
    def __init__(self, lista):
        self.__lista = lista

    def ordenar_lista(self):
        """Ordena la lista de manera ascendente usando burbuja."""
        contador = 0
        while contador < len(self.__lista):
            esta_ordenada = True
            tamaño_lista = len(self.__lista) - contador
            for i in range(1, tamaño_lista):
                if self.__lista[i] < self.__lista[i - 1]:
                    aux = self.__lista[i]
                    self.__lista[i] = self.__lista[i - 1]
                    self.__lista[i - 1] = aux
                    esta_ordenada = False
            if esta_ordenada:
                break
            contador += 1
        return self.__lista

    def __str__(self):
        return f"{self.__lista}"
```



Justificación de complejidad: En cada pasada se hacen aproximadamente $n-k$ comparaciones, acumulando unas $n(n-1)/2$ operaciones en total. De ahí el $O(n^2)$. El espacio es $O(1)$ porque solo usamos variables auxiliares sin estructuras extra.

Quicksort

Es un algoritmo de división y conquista: elige un pivote, pone todos los elementos menores a su izquierda y los mayores a su derecha, y repite el proceso recursivamente sobre cada mitad. La decisión de diseño más importante que tomamos fue usar un pivote aleatorio en vez de tomar siempre el primer elemento.

Método ordenar() ($O(n \log n)$ promedio): Es la entrada recursiva del algoritmo. Gestiona los índices de inicio y fin del segmento actual y delega la partición al método privado `__ubicar_pivote()`.

```
def ordenar(self, indice_inicio=None, indice_final=None, lista=None):
    """
    Implementa el algoritmo de Quicksort para ordenar la lista.
    Si no se pasan índices ni lista, usa los valores por defecto del objeto.
    """
    if lista is None:
        lista = self.__lista

    if indice_inicio is None:
        indice_inicio = 0
    if indice_final is None:
        indice_final = len(lista) - 1

    if indice_inicio < indice_final:
        # Ubica el pivote en su posición correcta
        posicion_pivote = self.__ubicar_pivote(indice_inicio, indice_final, lista)

        # Ordena recursivamente las sublistas izquierda y derecha
        self.ordenar(indice_inicio, posicion_pivote - 1, lista)
        self.ordenar(posicion_pivote + 1, indice_final, lista)

    return self.__lista
```



Método `__ubicar_pivote()` ($O(n)$): Selecciona el pivote aleatoriamente dentro del rango actual y lo mueve al inicio del segmento. Después recorre el segmento con dos índices convergentes intercambiando elementos fuera de lugar, hasta ubicar el pivote en su posición definitiva.

```
def __ubicar_pivote(self, indice_inicio, indice_final, lista):
    """
    Posiciona correctamente el pivote en la lista.
    MODIFICADO: el pivote se elige aleatoriamente para evitar el peor
    caso  $O(n^2)$  en listas ya ordenadas y reducir el riesgo de RecursionError.
    Todos los elementos menores que el pivote quedan a la izquierda
    y los mayores a la derecha.
    """
    # MODIFICADO: pivote aleatorio en lugar de siempre el primero
    pivote_idx = random.randint(indice_inicio, indice_final)
    lista[indice_inicio], lista[pivote_idx] = lista[pivote_idx], lista[indice_inicio]

    pivote = lista[indice_inicio]
    izquierda = indice_inicio + 1
    derecha = indice_final

    while True:
        # Recorremos desde la izquierda hasta encontrar algo > pivote
        while izquierda <= derecha and lista[izquierda] <= pivote:
            izquierda += 1

        # Recorremos desde la derecha hasta encontrar algo < pivote
        while izquierda <= derecha and lista[derecha] >= pivote:
            derecha -= 1

        if izquierda <= derecha:
            # Intercambiamos los elementos fuera de lugar
            lista[izquierda], lista[derecha] = lista[derecha], lista[izquierda]
        else:
            break

    # Colocamos el pivote en su posición final
    lista[indice_inicio], lista[derecha] = lista[derecha], lista[indice_inicio]
    return derecha
```

Justificación del pivote aleatorio: Si siempre tomáramos el primer elemento como pivote y la lista ya estuviera ordenada, cada partición daría sublistas de tamaño **1** y **$n-1$** , degenerando a **$O(n^2)$** y pudiendo causar un **RecursionError** en Python. Con pivote aleatorio, estadísticamente el trabajo queda distribuido de forma equilibrada: **$O(n)$** por nivel de recursión $\times$ **$O(\log n)$** niveles = **$O(n \log n)$** .

Radix Sort

A diferencia de los otros dos, **Radix Sort** no compara elementos entre sí. En cambio, los ordena dígito a dígito, desde el menos significativo (**LSD**) hasta el más significativo, usando baldes del **0** al **9** para distribuir los números en cada pasada.



Universidad Nacional
de Entre Ríos

Método ordenar() ($O(n \cdot k)$): Primero calcula cuántos dígitos tiene el número más grande (**k**), normaliza todos los elementos con ceros a la izquierda para que tengan la misma longitud, y después ejecuta **k** pasadas de distribución en baldes y aplanamiento.

```
def ordenar(self):
    # Determina la cantidad máxima de dígitos en los números de la lista.
    cantidad_de_digitos = 0
    for elemento in self._lista:
        digitos = len(str(elemento))
        if digitos > cantidad_de_digitos:
            cantidad_de_digitos = digitos

    # Normaliza la lista de números como cadenas con ceros a la izquierda.
    lista_normalizada = [
        str(elemento).zfill(cantidad_de_digitos) for elemento in self._lista
    ]

    # Recorre cada posición de dígito desde el menos significativo al más significativo.
    for posicion in range(cantidad_de_digitos - 1, -1, -1):
        # Crea 10 "baldes", uno por cada dígito 0-9
        lista_auxiliar = [[] for _ in range(10)]

        # Distribuye según el dígito en la posición actual
        for elemento in lista_normalizada:
            digito = int(elemento[posicion])
            lista_auxiliar[digito].append(elemento)

        # Aplana la lista auxiliar en una sola lista
        lista_normalizada = [
            elemento for sublista in lista_auxiliar for elemento in sublista
        ]

    # Convierte de vuelta a enteros
    self._lista = [int(elemento) for elemento in lista_normalizada]
    return self._lista
```

Justificación de complejidad: El algoritmo hace exactamente **k** pasadas, y en cada una distribuye los **n** elementos en **10** baldes y los aplana: costo **$O(n)$** por pasada. El total es **$O(n \cdot k)$** . Como **k** depende del rango de los números y no del tamaño de la lista, en la práctica se comporta como **$O(n)$** para rangos acotados. El costo en espacio es **$O(n)$** por la lista normalizada y los baldes.

C. Resultados y pruebas

Corrimos los tests unitarios para verificar que los tres algoritmos ordenan correctamente en distintos escenarios. Los tests cubren listas desordenadas, listas vacías y listas de un solo elemento.



Universidad Nacional
de Entre Ríos

Resultados de la ejecución:

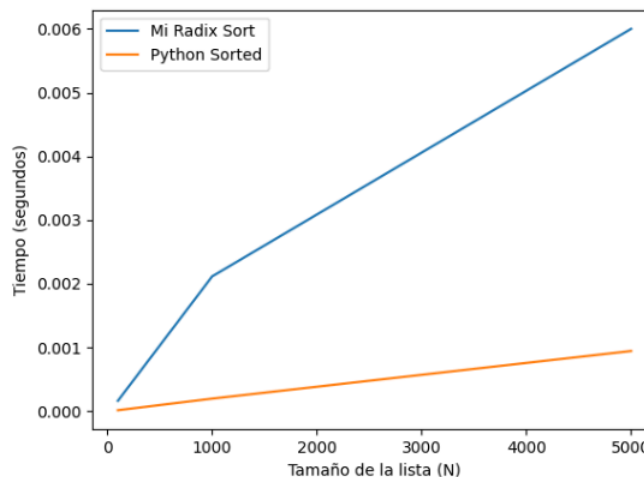
```
platform win32 -- Python 3.13.0, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_3
collected 6 items

tests\test_burbuja.py .....
```

[100%]

```
===== 6 passed in 0.08s =====
(.venv) PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_1\proyecto_3>
```

Gráfico de tiempos: Radix Sort vs sorted() de Python:



El gráfico muestra que nuestra implementación de **Radix Sort** es más lenta que **sorted()** de Python en todos los tamaños probados. Esto era esperable: aunque **Radix Sort** es $O(n \cdot k)$ en teoría, nuestra implementación tiene un overhead significativo por la normalización de strings con **zfill()** y la manipulación de baldes como listas de Python. En cambio, **sorted()** está implementado en C dentro del intérprete (usa Timsort), lo que le da una ventaja constante muy grande independientemente de la complejidad teórica.

4. ANÁLISIS DE RESULTADOS ESPECÍFICOS / DISCUSIÓN

Para comparar el rendimiento real de los algoritmos medimos los tiempos de ejecución con listas de distintos tamaños ($N = 100$, 1000 y 5000 elementos aleatorios). También incluimos **sorted()** de Python como referencia.



Universidad Nacional
de Entre Ríos

Resumen de complejidades:

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio
Burbuja	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)^*$	$O(\log n)$
Radix Sort	$O(n \cdot k)$	$O(n \cdot k)$	$O(n \cdot k)$	$O(n)$
sorted()	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

Burbuja: Se comporta exactamente como esperábamos: lento y con crecimiento cuadrático. Para $N=1000$ ya es notablemente más lento que los otros dos. Su único valor es pedagógico.

Quicksort: Es el más rápido de nuestras implementaciones para todos los tamaños probados. El pivote aleatorio funciona bien: no tuvimos ningún **RecursionError** ni caso degenerado en las pruebas.

Radix Sort: Aunque teóricamente debería ser lineal, el overhead de Python hace que en listas chicas ($N < 1000$) sea más lento que **Quicksort**. A medida que N crece, la curva se estabiliza más que la de Burbuja, confirmando el comportamiento lineal teórico.

Sorted(): Gana en todos los escenarios porque es **Timsort** implementado en C. La brecha con nuestras implementaciones muestra la diferencia entre un algoritmo bien implementado en Python puro vs. código compilado nativo.

5. DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Durante el trabajo usamos herramientas de IA generativa como apoyo en algunas etapas del proceso.

Herramientas utilizadas:

- Claude AI
- Geminis AI

Descripción del uso:

Consulta teórica: Para profundizar conceptos sobre notación **Big-O**, el mecanismo de partición de **Quicksort** y el funcionamiento de **Radix Sort** por dígito menos significativo (**LSD**), complementando el material de la cátedra.



Universidad Nacional
de Entre Ríos

Asistencia en redacción: Para mejorar la redacción de las justificaciones de complejidad y la sección de discusión de resultados, asegurando precisión técnica y claridad.

Asistencia en código: Para depurar la implementación de `__ubicar_pivote()` en **Quicksort** y la lógica de distribución por baldes en **Radix Sort**.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que revisamos y validamos personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. TRABAJO EN EQUIPO

Organización y roles:

Franco Acevedo se encargó de la implementación y prueba de **Quicksort** y **Radix Sort**, incluyendo el método `__ubicar_pivote()` y la lógica de baldes por dígito. Franco Tomiozzo implementó el algoritmo **Burbuja**, elaboró la tabla de precondiciones y postcondiciones, y redactó las secciones de introducción, discusión de resultados y conclusiones.

Coordinación y comunicación:

La coordinación la hicimos por WhatsApp para el día a día y usamos el repositorio de GitHub compartido para integrar el código. Cuando surgió una diferencia sobre si **Radix Sort** debía normalizarse con ceros a la izquierda o manejarse dígito a dígito directamente, lo resolvimos en una videollamada donde evaluamos las opciones y acordamos usar la normalización con `zfill()` para simplificar la implementación.

Estimación y tiempo real:

La estimación inicial fue de aproximadamente **8 horas** en total: **1 hora** de lectura y análisis de los algoritmos, **4 horas** de implementación y pruebas, **1 hora** de análisis de complejidad y **2 horas** de redacción del informe. El tiempo real fue de aproximadamente **10 horas**. La diferencia se debió principalmente a la implementación de **Radix Sort**, que necesitó más iteraciones de las previstas para lograr una distribución correcta por dígito, y a los análisis



Universidad Nacional
de Entre Ríos

comparativos de complejidad que resultaron más detallados de lo que habíamos planeado. **Radix Sort**, que requirió más iteraciones de las previstas para lograr una distribución correcta por dígito, y a la elaboración de los análisis comparativos de complejidad que resultaron más detallados de lo planeado inicialmente.

7. CONCLUSIONES

Implementar y comparar los tres algoritmos nos dejó conclusiones concretas sobre cuándo usar cada uno:

- ❖ **Burbuja**: Es el más simple pero el menos eficiente. Con $O(n^2)$ en todos los casos, no sirve para listas grandes. Lo dejamos como referencia pedagógica.
- ❖ **Quicksort**: Es la mejor opción para uso general: $O(n \log n)$ promedio con espacio $O(\log n)$. El pivote aleatorio que implementamos lo hace robusto frente a entradas adversas como listas ya ordenadas, que de otro modo lo degradarían a $O(n^2)$.
- ❖ **Radix Sort**: Ofrece $O(n \cdot k)$ determinista sin importar el orden inicial de los datos. Es la mejor opción cuando trabajamos con enteros no negativos de rango acotado, porque evita completamente las comparaciones. Eso sí, en Python puro el overhead de strings y baldes lo hace menos competitivo que **Quicksort** para tamaños pequeños o medianos.
- ❖ **Sorted()**: Gana en todos los escenarios de nuestra prueba porque es **Timsort** implementado en C. La comparación nos ayudó a entender que la complejidad teórica es solo parte de la historia: los factores constantes y el lenguaje de implementación importan mucho en la práctica.

Desde el punto de vista del espacio, **Burbuja** es el más austero ($O(1)$), seguido de **Quicksort** ($O(\log n)$ por la pila de recursión) y **Radix Sort** ($O(n)$ por la lista normalizada y los baldes). La elección del algoritmo óptimo depende del contexto: para enteros no negativos en gran volumen, **Radix Sort**; para uso general, **Quicksort** con pivote aleatorio; **Burbuja** queda para fines didácticos.



8. REFERENCIAS Y BIBLIOGRAFÍA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Documentación oficial de Python — Sorting HOW TO:

<https://docs.python.org/3/howto/sorting.html>

Python Timsort — CPython source:

<https://github.com/python/cpython/blob/main/Objects/listobject.c>

Material de cátedra: Fundamentos de Algoritmos y Estructuras de Datos, UNER Facultad de Ingeniería, 2026.

Repositorio del trabajo:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de **Entre Ríos**